Certifying global optima in non-convex optimization problems using deep neural networks

Charles Gulian *charlesgulian@berkeley.edu*
Industrial Engineering & Operations Research Dept.
University of California, Berkeley

Ying Chen *ying-chen@berkeley.edu*
Industrial Engineering & Operations Research Dept.
University of California, Berkeley

Javad Lavaei *lavaei@berkeley.edu*
Industrial Engineering & Operations Research Dept.
University of California, Berkeley

Reviewed on OpenReview: *https://openreview.net/forum?id=XXXX*

Abstract

Solving non-convex optimization problems in real-time is a ubiquitous challenge. Most fast solvers are not guaranteed to find globally optimal solutions to non-convex problems. On the other hand, convexification techniques (such as semidefinite programming relaxations) can globally solve large classes of non-convex problems but scale poorly with problem size. We propose a certification meta-framework that bridges this gap by combining machine learning and convexification techniques. Unlike approximate solvers that offer probabilistic guarantees, our approach provides deterministic certificates for the value function of a chosen relaxation. We use neural networks to approximate the value function of a parametric non-convex problem by generating data from exact or approximate convex relaxations offline. The trained network's predictions may then be used online to check optimality of solutions obtained via other solvers in real-time. We demonstrate our approach on several non-convex problems, including AC-OPF, which is the motivating application of this work.

1 Introduction

Online non-convex optimization is ubiquitous in cyber-physical systems engineering. Power system operators solve AC optimal power flow (AC-OPF) to balance time-varying load and generation (**?**). Autonomous vehicles optimize routes to avoid collisions in time-varying environments (**?**). Digital signals are recovered from time-varying communication channels (**?**). The list of applications goes on.

Unlike in convex optimization, global optimality in non-convex problems may not be certified from first-order information such as the KKT conditions. It is therefore challenging to determine whether a solution is optimal. In fact, non-convex optimization can be NP-hard (**?**). A key challenge is the possible presence of multiple local minima where local search methods can get stuck. Heuristic algorithms such as simulated annealing and evolutionary search deal with this challenge by exploring the solution space more broadly (**??**). Machine learning-based solvers have also been used to obtain good solutions to non-convex problems, including AC-OPF (**?**). However, these methods do not guarantee or certify global optimality.

On the other hand, convexification techniques like semidefinite relaxation provide a way to globally solve many non-convex problems. Convexification replaces non-convex objectives or constraints with convex outer-approximations, often in higher-dimensional spaces. Convex relaxations are solvable in polynomial time and provide global lower bounds for the optimal value of the original non-convex problem (**?**). However,

solving a convex relaxation can be exponentially slower than solving the original problem with local search or machine learning-based solvers, as convexification may introduce many more variables and constraints. Therefore, despite their power, these approaches scale poorly with problem size and are impractical to use in real-time applications.

We propose a framework which draws upon the strengths of both machine learning and convexification techniques to assist in online non-convex optimization. Specifically, we use neural networks to approximate the value functions of convex relaxations of non-convex problems. Convexification is used offline to obtain training data for learning the value function, while the trained neural network is used online to certify optimality of feasible solutions obtained by fast solvers. Our approach thus distills the power of convexification into *neural network optimality certificates* that can be deployed in real-time application settings.

1.1 Contributions

We make the following contributions in this work.

- We propose a novel machine learning framework that uses neural networks as optimality certificates for non-convex optimization problems.
- We use convexification to solve non-convex problems and obtain training data for learning the value functions of non-convex problems.
- We exploit input-convex and scale-invariant neural network architectures to improve performance in specific problem settings.
- We provide new performance guarantees for input-convex neural networks, which have implications for the robustness of our optimality certificates.
- We demonstrate the viability of our framework in several realistic experiments.

1.2 Paper Outline

In Section **??**, we give an overview of relevant background topics. In Section **??**, we describe our framework in the context of several powerful convexification methods, and demonstrate our approach in a simple problem setting. In Section **??**, we highlight results from several experiments. We conclude in Section **??**.

2 Background Overview

Machine learning-based solvers have been proposed in a growing number of application areas, including power systems optimization, digital communications, and integer programming (**???**). Machine learning-based solvers often achieve good performance on their respective target problems, and have been shown to be faster and more accurate than local search and heuristic algorithms in many cases. They therefore appear to be promising alternatives to conventional solution methods. However, like the methods which they seek to replace, machine learning-based solvers suffer from the inability to guarantee or certify optimality (or even feasibility) of obtained solutions. As a result, machine learning-based solvers have not been widely integrated into safety-critical applications (**?**).

This is largely due to the inherent challenges of non-convex optimization. Certifying optimality in NP-hard problems can be just as hard as finding optimal solutions to begin with. However, for certain non-convex problems – namely, those which may be *convexified* – optimal solutions may be found and certified in polynomial time.

Convexification involves finding an “exact" convex relaxation of a non-convex problem – i.e., a convex relaxation whose optimal value and optimal solution are the same as the original problem's. Convexification techniques have been developed for many non-convex problems, including AC-OPF and quadratically-constrained quadratic programming (QCQP), as well as combinatorial optimization problems (**???**). Sufficient conditions for the exactness of semidefinite programming (SDP) relaxations have been found for large classes of non-convex problems (**?**). Similar conditions have been found for non-convex QCQPs, including constraint qualifications like the S-lemma and bounds on the maximum rank of optimal solutions of SDP relaxations

(**??**). Furthermore, the Lasserre hierarchy even provides a systematic procedure for constructing arbitrarily tight SDP relaxations of polynomial optimization problems (**?**) (see Section **??**). Convexification is therefore a powerful and general technique for solving non-convex problems.

However, a common theme among convexification methods is that convexified problems may be orders of magnitude larger than the original non-convex problem, making these techniques cumbersome to use in online application settings. For instance, for 0-1 programming problems in $\mathbb{R}^n$, the order-d Sherali-Adams hierarchy LP has $\sum_{i=1}^{d} \binom{n}{i}$ variables. The SDP relaxation of a QCQP has $n(n+1)/2$ variables compared to n variables in the original problem. For an arbitrary polynomial optimization problem in $\mathbb{R}^n$, the decision variable of the order-d Lasserre hierarchy SDP is a $\binom{n+d}{d} \times \binom{n+d}{d}$ symmetric positive semidefinite matrix. However, we claim that the power of convexification may be harnessed to *learn* the value functions of non-convex problems *offline* and thus provide optimality certificates. Indeed, this is the central idea of our framework for developing neural network optimality certificates.

Recent works have also begun exploring connections between neural networks and convexification techniques. For instance, **?** proposed GloptiNets, a framework that leverages the Kernel Sum-of-Squares (KSOS) method and neural network architecture and training algorithms to efficiently solve non-convex problems on the unit hyper-cube. GloptiNets serve as an approximate solver that produces probabilistic optimality certificates at user-specified confidence levels. While powerful, it remains a solver for high-order relaxations and can be computationally intensive (slow) for real-time applications. In contrast, our approach uses convexification offline to learn value function surrogates, which enable approximate optimality certification in real-time. In fact, efficient convexification techniques such as GloptiNets fit neatly within our framework as necessary ingredients for generating training data to learn value function surrogates.

We review additional background topics in Appendix **??**. One background topic that we have omitted in this brief overview is *special neural network architectures*, which we use to improve the performance and robustness of our framework. For many non-convex problems, learning to certify optimality amounts to learning to approximate a convex value function, for which input-convex neural networks are well suited (**?**).

3 Our Framework

We propose this method as a certification meta-framework. It is distinct from a standalone solver in that it is not tied to a single convexification technique. Any convexification trick, valid inequality, or instance-specific lower bound methodology from the optimization literature can be applied within our framework. The core novelty is the mechanism that allows us to take any computationally expensive convex relaxation—whether exact or approximate—and convert it into a millisecond-scale verified proxy. This bridges the gap between high-dimensional convex geometry and real-time control.

Consider a parametric optimization problem:

$$\begin{aligned} v(p) := \min_{x \in \mathbb{R}^n} \quad & f(x;p) \\ \text{s.t.} \quad & g(x;p) \leq 0 \end{aligned} \tag{1}$$

in which the parameter $p \in \Phi \subseteq \mathbb{R}^d$ (where Φ is an arbitrary parameter set). We assume that equation **??** is feasible for all $p \in \Phi$. The value function $v : \mathbb{R}^d \to \mathbb{R}$ expresses the globally optimal objective value as a function of the parameter p. The objective function $f(x;p)$ and/or the constraints $g(x;p)$ may be non-convex, so equation **??** may be a non-convex problem.

The value function may be used to test optimality of arbitrary feasible solutions. Given $\hat{x} \in \mathbb{R}^n$ that is feasible for equation **??**, if the cost $f(\hat{x};p) = v(p)$ then $\hat{x}$ is optimal; otherwise, if $f(\hat{x};p) > v(p)$, it is not.

Furthermore, *approximations* of the value function may be used to test *near*-optimality of arbitrary feasible solutions. Let $\hat{v}(p)$ be a function that approximates $v(p)$ over Φ. If one can bound the approximation error of $\hat{v}(p)$ over Φ, then they may also bound the optimality gap $f(\hat{x};p) - v(p)$ by comparing $f(\hat{x};p)$ to $\hat{v}(p)$.

These observations motivate our work in this paper: we propose a machine learning framework to approximate the value functions of non-convex optimization problems for the purpose of approximate optimality certification. This framework may be especially beneficial in online settings, where obtaining exact optimality certificates may be expensive or impractical.

In particular, we propose to train a neural network to approximate $v(p)$ over the parameter set Φ. We solve many instances of equation **??** for different $p \in \Phi$ to get training samples $(p_i, v(p_i))$ with i ranging from 1 to N. The neural network $\hat{v}_\theta(p)$ is trained to minimize the mean-squared error loss:

$$\min_{\theta} \ \frac{1}{N} \sum_{i=1}^{N} (v(p_i) - \hat{v}_\theta(p_i))^2 \tag{2}$$

where θ represents the network's weights and biases. If the trained neural network $\hat{v}_\theta(p)$ achieves low approximation error over the full parameter set Φ, we may use it to certify near-optimality of solutions obtained via local search methods or other solvers. (Hereafter, we drop the 'θ' subscript notation for $\hat{v}(p)$ when referring to trained neural networks.)

The central challenge of learning to approximate the value function is developing a rich training data set. This is because evaluating $v(p)$ many times may be expensive, as it involves repeatedly solving a non-convex optimization problem. This challenge may not always be easily overcome. However, for a large class of non-convex problems, convexification techniques allow us to find globally optimal solutions relatively efficiently. Therefore, the use of convexification techniques is a key element of our framework.

We now describe the role of convexification techniques within our framework. Consider a convex relaxation of problem equation **??**:

$$\begin{aligned} \tilde{v}(p) := \min_{x \in \mathbb{R}^n, \ u \in \mathbb{R}^k} \ & \tilde{f}(x, u; p) \\ \text{s.t.} \quad & \tilde{g}(x, u; p) \leq 0 \end{aligned} \tag{3}$$

where $\tilde{f}(x, u; p)$ and $\tilde{g}(x, u; p)$ are convex outer-approximations of $f(x; p)$ and $g(x; p)$, respectively, and $u \in \mathbb{R}^k$ are (optional) additional variables introduced to lift or convexify the problem. Hereafter, we use $\tilde{v}(p)$ to denote the value function of any convex relaxation. Since equation **??** is a convex relaxation of equation **??**, we have that $\tilde{v}(p) \leq v(p)$ for all $p \in \Phi$. Therefore, solving equation **??** always yields a global lower bound on $v(p)$. When $\tilde{v}(p) = v(p)$, we say the relaxation is "exact."

One may evaluate $v(p)$ in polynomial time by solving an *exact* convex relaxation of equation **??**. This key insight enables our framework to be applied to any non-convex problem for which an exact or tight convex relaxation exists. By finding an exact convex relaxation of equation **??** for (almost) any $p \in \Phi$, one may solve problem instances of equation **??** relatively efficiently to develop a training data set for the learning problem equation **??**.

As an important caveat, we note that convexification enables exact evaluation of $v(p)$ at the potential expense of slower computation due to the much larger size of the convexified problem. This is precisely why we propose to use convexification techniques *offline* to develop training data sets, and use neural networks *online* to provide approximate optimality certificates.

3.1 Convexification Techniques

Convexification is a powerful approach for globally solving non-convex problems. The basic idea of convexification is to replace non-convex objectives or constraints with convex outer-approximations. The resulting convex problem is a relaxation of the original non-convex problem, and may be solved in polynomial time. The optimal value of the convex relaxation is a lower bound on the optimal value of the original problem. In some cases, an optimal solution of the original problem may be recovered from the convex relaxation, meaning the original problem may also be solved in polynomial time.

3.1.1 Structured QCQP

Our method is highly compatible with structured QCQP, which is a class of QCQPs that exhibit specific structure – either in the objective, the constraints, or both – that can be exploited to make the problem easier to solve or analyze.

Many non-convex problems, including integer programming problems, may be exactly or approximately represented as polynomial optimization problems (POPs). A generic POP can be written as

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \; & f(x) = \sum_{\alpha} f_\alpha x^\alpha \\ \text{s.t.} \; & g_i(x) = \sum_{\alpha} g_{i,\alpha} x^\alpha \geq 0, \quad i = 1, \ldots, m \end{aligned} \tag{4}$$

where both the objective $f(x)$ and constraints $g_i(x)$ are polynomials in $x \in \mathbb{R}^n$. We use the multi-index notation $x^\alpha := x_1^{\alpha_1} \cdots x_n^{\alpha_n}$ for $x \in \mathbb{R}^n$ and $\alpha \in \mathbb{N}^n$.

Furthermore, a generic POP may be re-formulated via quadrification (**??**) as a QCQP of the form

$$\begin{aligned} \min_{y\in\mathbb{R}^k} \; & y^T M_0 y \\ \text{s.t.} \; & y^T M_i y \geq 0, \quad i = 1, 2, ..., l. \end{aligned} \tag{5}$$

Generally, this requires introducing new variables and constraints to re-write higher degree (> 2) monomials as quadratic terms. For example, the constraint '$3x^6 - 2x^2 \geq 0$' may be equivalently represented by the constraints '$y = x^2$, $z = y^2$, and $3yz - 2y \geq 0$.' As a result, the equivalent QCQP may be much larger than the original POP in terms of the number of variables ($k \gg n$) and constraints ($l \gg m$).

Non-convex QCQPs are therefore a very broad class of optimization problem. Importantly, their simple structure has allowed for detailed study of QCQP convexification. In particular, a number of structural characterizations have been developed which are sufficient for exactness of the SDP relaxation of a QCQP:

$$\begin{aligned} \min_{Y\in\mathbb{R}^{n\times n}} \; & \langle M_0, Y \rangle \\ \text{s.t.} \; & \langle M_i, Y \rangle \geq 0, \quad i = 1, 2, ..., l \\ & Y \succeq 0 \end{aligned} \tag{6}$$

Examples of such structural characterizations include conditions on the number of constraints in equation **??**, such as the S-lemma (**?**) and a number of more general results on low-rank SDP relaxations (**?**). For example, the S-lemma proves the exactness of an SDP relaxation of a QCQP with a single quadratic constraint. Other examples include conditions on the underlying graph structure of the QCQP, as in **?** and **?**. For instance, if the underlying graph of a QCQP is acyclic, then its SDP relaxation must be exact. QCQPs with exact SDP relaxations may be solved efficiently, even if they have disconnected feasible regions or multiple local minima.

Additionally, exactness of the SDP relaxation of a QCQP may be ascertained not only *a priori* via structural characterizations but also *a posteriori* by proving feasibility of the optimal solution in the original problem. For example, exactness may ascertained from the rank of an optimal solution (**?**). If Y^* is optimal for the SDP relaxation equation **??** and $\text{rank}(Y^*) = 1$, then there exists a vector $x^* \in \mathbb{R}^n$ such that $Y^* = y^* y^{*T}$ and that is feasible (and thus optimal) for equation **??**. Therefore, whenever equation **??** is an exact relaxation of equation **??**, the minimum values of the two problems coincide and a globally optimal solution of equation **??** may be recovered from a solution of equation **??**. If there are multiple optimal solutions to equation **??**, it is possible that an optimal solution Y^* obtained from an SDP solver is not rank-1 even though a rank-1 solution exists, in which case the optimal values agree anyways.

3.1.2 QCQPs with Convex Value Functions

Consider a QCQP parameterized by right-hand side uncertainty:

$$\begin{aligned} v(b) := \min_{y\in\mathbb{R}^k} \; & y^T M_0 y \\ \text{s.t.} \; & y^T M_i y \geq b_i, \quad i = 1, 2, ..., l \end{aligned} \tag{7}$$

As a minor point, we observe that if the parametric QCQP equation **??** admits an exact convex relaxation for all parameters $b \in \Phi$ where $\Phi \subseteq \mathbb{R}^m$ is some convex parameter set, then the value function $v(b)$ is convex over Φ. The proof of this statement is trivial and follows the well-known argument for convexity of the value function of convex problems parameterized by right-hand side uncertainty. However, the result is useful in practice. For instance, load-parameterized AC-OPF falls into this category of parametric QCQPs. The result also has implications for the *types* of neural networks that can approximate the value functions of QCQPs.

3.1.3 Sum-of-Squares and Lasserre Hierarchy

Testing whether a polynomial $f(x)$ is globally non-negative ($f(x) \geq 0, \forall x$) is NP-hard (**?**). However, the sum of squares (SOS) decomposition provides a sufficient condition for non-negativity (**?**). A polynomial $p(x)$ is an SOS if there exist polynomials $q_i(x)$ such that $p(x) = \sum_{i=1}^{m} q_i^2(x)$.

Let $\Sigma[x]$ be the set of SOS polynomials, which forms a convex cone, and $\Sigma[x]_d$ be the set of SOS polynomials of degree at most $2d$. A polynomial $\sigma(x) \in \Sigma_d[x]$ if it is of the form

$$\sigma(x) = \sum_k \left(\sum_{|\alpha| \leqslant d} p_{k,\alpha} x^\alpha \right)^2 \text{ where } p_{k,\alpha} \in \mathbb{R}. \tag{8}$$

This is equivalent to the existence of $(\varphi_{\alpha,\beta})_{|\alpha|,|\beta| \leqslant d} \succcurlyeq 0$ such that $\sum_{|\alpha| \leqslant 2d} \sigma_\alpha x^\alpha = \sum_{|\alpha|,|\beta| \leqslant d} \varphi_{\alpha,\beta} x^{\alpha+\beta}$.

Let $\mathbf{S} := \{x \in \mathbb{R}^n \mid g_1(x) \geq 0, \ldots, g_m(x) \geq 0\}$, $\mathscr{M}(\mathbf{S})$ be the vector space of finite signed Borel measures supported on $\mathbf{S}$, and $\mathscr{M}_+(\mathbf{S})$ denote the cone of nonnegative elements of $\mathscr{M}(\mathbf{S})$. Problem equation **??** can be reformulated as a linear program over probability measures (**?**):

$$\min_{\mu \in \mathscr{M}_+(\mathbf{S})} \left\{ \int_{\mathbf{S}} f \, \mathrm{d}\mu : \int_S \, \mathrm{d}\mu = 1 \right\}. \tag{9}$$

To make this reformulation tractable, we introduce the r-th order moment matrix $\mathbf{M}_r(y)$ associated with a sequence y, and the r-th order localizing matrix $\mathbf{M}_r(g_j y)$ associated with y and constraint g_j:

$$\begin{aligned} \mathbf{M}_r(y) &:= (y_{\alpha+\beta})_{|\alpha|,|\beta| \leqslant r} \\ \mathbf{M}_r(g_j y) &:= (\sum_\gamma g_{i,\gamma} y_{\alpha+\beta+\gamma})_{|\alpha|,|\beta| \leqslant r} \end{aligned} \tag{10}$$

We construct a hierarchy of moment relaxations for problem equation **??**, as $\mu \in \mathscr{M}_+(\mathbf{S})$ can be relaxed to $\mathbf{M}_{r-d_j}(g_j y) \succeq 0$, for all $j = 0, \ldots, m$, and all $r \geq d_j = \lceil \deg(g_j)/2 \rceil$. Here, $\lceil \cdot \rceil$ denotes the ceiling function, and $\deg(\cdot)$ denotes the degree of a polynomial function.

Letting $r_{\min} := \max\{\lceil \deg(f)/2 \rceil, d_1, \ldots, d_m\}$, for $r \geq r_{\min}$ of the hierarchy, we construct the order-r moment SDP:

$$\begin{aligned} \min_{y} \quad & L_y(f) \\ \text{s.t.} \quad & \mathbf{M}_r(y) \succeq 0 \\ & \mathbf{M}_{r-d_j}(g_j y) \succeq 0, \quad j = 1, \ldots, m \\ & y_0 = 1 \end{aligned} \tag{11}$$

where $L_y(f) := \sum_\alpha f_\alpha y^\alpha$, and $y \in \mathbb{R}^{\frac{(n+2r)\ldots(n+1)}{(2r)!}}$. The dual of problem equation **??** is thus the following optimization problem over SOS polynomials of order r:

$$\begin{aligned} \max_{\lambda, \sigma} \quad & \lambda \\ \text{s.t.} \quad & f - \lambda = \sigma_0 + \textstyle\sum_{k=1}^m \sigma_k g_k \\ & \lambda \in \mathbb{R}, \sigma_0 \in \Sigma_r[x], \\ & \sigma_i \in \Sigma_{r-d_j}[x], \quad j = 1, \ldots, m. \end{aligned} \tag{12}$$

The Lasserre hierarchy, presented here, thus provides a sequence of SDP relaxations for solving the POP equation **??**. By defining $\mathcal{M}(\mathfrak{g})$ to be the quadratic module generated by $\mathfrak{g}$, the following theorem provides a powerful condition for λ to be a lower bound on f.

Theorem 3.1 (Putinar's Positivstellensatz (**?**))**.** *Assume that there exists $N > 0$ such that $N - \|x\|_2^2 \in \mathcal{M}(\mathfrak{g})$ (Archimedean's condition). If f is positive on $\mathbf{S}$, then*

$$f = \sigma_0 + \sigma_1 g_1 + \cdots + \sigma_m g_m, \tag{13}$$

where $\sigma_0, \ldots, \sigma_m$ are SOS.

Under the Archimedean condition, the hierarchy converges to the global optimum of the original problem as $r \to \infty$ (**?**). Importantly, the relaxation error is asymptotically bounded by $\mathcal{O}(1/r)$. For specific problem classes, such as those with a ball constraint, there is often no duality gap even at lower relaxation levels (**?**). In cases where basic relaxations ($r = 1$) are insufficient or inexact, our framework naturally incorporates higher levels of the Lasserre hierarchy to tighten the bounds.

This flexibility allows for a strategic trade-off: one may choose a higher-order relaxation ($r \geq 2$) to reduce the relaxation gap $v(p) - \tilde{v}(p)$ and improve certificate quality, at the expense of increased offline computational cost for data generation. For instance, **?** demonstrate that while second-order relaxations for AC-OPF are tighter than first-order ones, they are significantly more expensive to solve. Our framework is agnostic to the chosen hierarchy level: it simply learns the value function $\tilde{v}(p)$ of a particular relaxation, which still provides a lower bound on $v(p)$ even when the relaxation is not exact.

3.2 Illustrative Example

Let us illustrate our framework in a simple problem setting. Consider the parametric QCQP over $x \in \mathbb{R}^2$:

$$\begin{aligned} v(p) = \min_{x \in \mathbb{R}^2} \ & (x_1 - p_1)^2 + (x_2 - p_2)^2 \\ \text{s.t. } & x_1 x_2 \geq 1 \end{aligned} \tag{14}$$

In words, the problem is to find a point of minimum squared distance to $p \in \mathbb{R}^2$ satisfying $x_1 x_2 \geq 1$. The quadratic constraint yields a non-convex feasible set with two disconnected regions. Therefore, any instance of this problem has two local minima.

As illustrated in Figure **??**, the convergence behavior of local search methods depends strongly on the initial point used in the search. Some initial points converge to the global minimum while others converge to the spurious local minimum. A naive strategy would be to try multiple initial points and keep the best solution found. However, without a principled strategy for selecting initial points, it is not clear whether this procedure will find the global minimum.

Consider solving the SDP relaxaton of problem equation **??**. First, we re-write equation **??** in compact matrix notation:

$$\begin{aligned} v(p) = \min_{y \in \mathbb{R}^3} \ & y^T M_0 y \\ \text{s.t. } & y^T M_1 y \geq 1 \\ & y_1 = 1 \end{aligned} \tag{15}$$

with $y := [1 \ \ x^T]^T$ and

$$M_0 = \begin{bmatrix} \|p\|_2^2 & -p_1 & -p_2 \\ -p_1 & 1 & 0 \\ -p_2 & 0 & 1 \end{bmatrix}, \quad M_1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & 0 \end{bmatrix}.$$

The SDP relaxation of equation **??** is:

$$\begin{aligned} \tilde{v}(p) = \min \ & \langle M_0, Y \rangle \\ \text{s.t. } & \langle M_1, Y \rangle \geq 1 \\ & Y_{11} = 1 \\ & Y \succeq 0 \end{aligned}$$

The SDP relaxation is exact for all $p \in \mathbb{R}^2$ because equation **??** has only one quadratic inequality constraint (see S-lemma; **?**).

Finally, we demonstrate the central proposition of this work: to train a neural network to approximate the value function of a non-convex optimization problem. We solved the SDP relaxation equation **??** for 5000 instances of p selected uniformly at random over $\Phi = [-5, 5] \times [-5, 5]$ to obtain a data set of $(p, v(p))$ pairs. The trained neural network predictions are shown in Figure **??**. The neural network achieved **0.01**% **average** / **0.78**% **maximum** prediction error over the test data set of 1500 points. The trained neural network can thus reliably certify near-optimality of solutions of equation **??**.

4 Experiments

We now present results from several experiments in which we applied our approach to non-convex optimization problems from power systems, digital communications, and integer programming. The purpose of these experiments is to demonstrate the viability of our framework in diverse problem settings. Additional information and details may be found in Appendix **??**.

4.1 Input-Convex and Scale-Invariant Neural Networks

In each experiment, we trained neural networks to approximate the value functions of parametric optimization problems. In some cases, the value functions possess geometric properties like convexity and scale invariance (positive homogeneity) that may also be enforced in the neural networks, for example, by constraining the weights and biases of the network to take certain values (see Appendix **??**).

While our framework does not require convex or scale invariant value functions, these properties may be exploited to provide better performance guarantees. In particular, input-convex neural networks (ICNNs) (**?**) have relatively strong performance guarantees compared to generic deep neural networks (DNNs) for approximating convex functions **?**. Throughout our experiments, we explored ICNNs' ability to approximate convex value functions. We also prove novel error bounds and performance guarantees for ICNNs in Appendix **??**.

4.2 Power Systems Optimization

The alternating current optimal power flow (AC-OPF) problem is solved by power system operators approximately every 5 minutes to minimize cost while balancing demand and generation. It is one of the most fundamental optimization problems in power systems. AC-OPF explicitly represents the steady-state physics of AC electricity networks, which are expressed as nonlinear equations relating nodal voltages and power injections (see formulation in Appendix **??**). These yield quadratic equality constraints in the formulation, which makes AC-OPF a special case of non-convex QCQP.

? derive a sufficient condition for exactness of the SDP relaxation of AC-OPF, which holds for many real-world power systems. However, solving the SDP relaxation of AC-OPF is often much slower than using machine learning-based solvers or local search methods, especially in large power systems. SDP relaxation is thus seldom used in real-time power system operations. However, by learning to approximate the value function of the SDP relaxation of AC-OPF, one may obtain optimality certificates for faster solvers which lack optimality guarantees on their own. This is the motivation for our experiment.

For several IEEE test power systems, we trained DNNs and ICNNs to approximate the value function of AC-OPF parameterized by real and reactive load at each bus. We obtained training and testing data by solving exact SDP relaxations of AC-OPF. The DNN and ICNN models achieved very low approximation error in our experiments. Maximum error was less than 2-3% in most experiments, and average error was near-zero. Furthermore, the approximation error was relatively consistent across test cases of widely varying size and data dimension, which indicates that our approach can scale well to larger power systems. Furthermore, our timing comparison results (Table **??**) show unsurprisingly that neural network inference is orders of magnitude faster than solving AC-OPF via convexification or local search. Overall, the results suggest that neural networks can quickly and reliably certify optimality of AC-OPF solutions.

4.3 Digital Communications

The MIMO detection problem, which is an instance of QCQP with complex variables, arises in many signal processing applications. Typically, MIMO detection must be solved for rapidly time-varying signal and channel parameters. Although convexification methods like SDP relaxation have been used to solve MIMO detection, they are typically too slow to be used in practice. Several heuristics have been developed to approximately (sub-optimally) solve MIMO detection in realistic online settings. In **?**, the authors developed a machine learning-based solver for the MIMO detection problem, which achieved good performance. However,

Table 1: Results of AC-OPF experiments. Value function approximation error for DNNs and ICNNs for different IEEE cases and experiment configurations. Results with $< 0.01\%$ error are marked with a long dash "—."

IEEE Case	Data Dim.	# Samples	Avg. Pct. Error		Max. Pct. Error	
			DNN	ICNN	DNN	ICNN
`ieee9`	1	5000	—	—	0.02%	0.01%
	3	5000	—	—	0.55%	1.25%
	6	10000	0.04%	—	2.22%	4.19%
`ieee14`	3	5000	—	—	0.54%	1.62%
	6	10000	—	—	0.53%	2.43%
	10	20000	—	—	0.21%	0.41%
`ieee30`	6	10000	—	—	0.39%	1.04%
	10	20000	—	—	0.26%	0.90%
	25	50000	—	—	0.13%	0.58%
`ieee118`	10	20000	0.03%	—	0.96%	0.29%
	25	50000	—	—	0.13%	0.53%
`ieee300`	60	30000	—	—	0.55%	0.53%

Table 2: Time to solve or predict the optimal cost of 100 AC-OPF instances of `ieee300`. Experiments were conducted on a 2024 MacBook Air equipped with an 8-core Apple M3 chip and 8 GB of unified memory.

Problem	Software	Time
AC-OPF SDP	CVXOPT	30m 55s
AC-OPF	IPOPT	9m 3s
DNN Inference	PyTorch	0.1s

heuristic and machine learning-based approaches lack optimality guarantees and may benefit from neural network optimality certificates.

We solved 5000 instances of the SDP relaxation of MIMO detection with randomly generated noisy signals $\bar{y}$. We certified the exactness of each SDP instance by testing whether a rank-1 solution was obtained. We trained a DNN to approximate the value function using the Adam optimizer with a learning rate of 0.01 over 3000 epochs. The trained DNN achieved near-zero training loss, and **0.3**% **average / 2.5**% **maximum** error over the test data set of 1500 points. The trained DNN can therefore reliably certify optimality of detected digital signals.

4.4 Integer Programming

In this experiment, we trained neural networks to certify optimality of the 0-1 knapsack problem parameterized by the cost vector $c \in \mathbb{R}^n_+$. We solved thousands of problem instances using Gurobi's mixed-integer linear program solver (**?**) to obtain data sets for our experiments. We trained a variety of neural network architectures, including DNNs, ICNNs, and scale-invariant and convex scale-invariant networks (SINNs and CSINNs) to approximate the value function. We measured the average and maximum approximation error of each network in each experiment. The results are summarized in Table **??**.

The average performance of ICNNs and CSINNs was substantially better than DNNs and SINNs, indicating that special neural network architectures can reduce overfitting and improve performance. The good performance of ICNNs and CSINNs in our experiments highlights the potential to integrate neural networks into other solvers. For example, neural networks may be used to certify optimality of linear relaxations of

Table 3: Results of 0-1 knapsack experiments. Each block shows the results of a different set of experiments.

10 Item Knapsack: data dim. = 5, # samples = 50000

Network	Avg. Pct. Error	Max. Pct. Error
DNN	0.27%	2.29%
ICNN	0.02%	2.12%
SINN	0.30%	3.87%
CSINN	**0.01%**	**0.56%**

30 Item Knapsack: data dim. = 8, # samples = 80000

Network	Avg. Pct. Error	Max. Pct. Error
DNN	1.32%	8.88%
ICNN	**0.10%**	**0.92%**
SINN	1.20%	15.67%
CSINN	0.50%	5.02%

integer programs, or speed branch-and-bound by pruning branches whose objective function value exceeds the network's prediction by a pre-determined threshold.

5 Conclusion

In this work, we proposed an innovative framework for real-time optimality certification in non-convex optimization tasks. Our framework leverages convexification techniques and machine learning to learn the optimal value function of parametric optimization problems. By functioning as a meta-framework, our method is compatible with various convexification schemes, including the Lasserre hierarchy for general polynomial optimization, as well as various fast solvers. By training neural networks to approximate the optimal value of a convex relaxation, we shift the computational burden from real-time optimization to offline training. Under suitable conditions, the convex relaxation provides a tight lower bound on the original non-convex problem, so candidate solutions that are sufficiently close to the predicted optimal value are assured to be near-globally optimal. Our proposed framework not only expands the applicability of convexification methods to online optimization, but also offers a new conceptual approach to apply learning-based performance guarantees to fast heuristic solvers. Future work will focus on refining machine learning-based optimality certification and extending this approach to more diverse classes of non-convex problems.

6 Preparing PostScript or PDF files

Please prepare PostScript or PDF files with paper size "US Letter", and not, for example, "A4". The -t letter option on dvips will produce US Letter files.

Consider directly generating PDF files using `pdflatex` (especially if you are a MiKTeX user). PDF figures must be substituted for EPS figures, however.

Otherwise, please generate your PostScript and PDF files with the following commands:

```
dvips mypaper.dvi -t letter -Ppdf -G0 -o mypaper.ps
ps2pdf mypaper.ps mypaper.pdf
```

6.1 Margins in LaTeX

Most of the margin problems come from figures positioned by hand using `\special` or other commands. We suggest using the command `\includegraphics` from the graphicx package. Always specify the figure width as a multiple of the line width as in the example below using .eps graphics

```
\usepackage[dvips]{graphicx} ...
```

```
\includegraphics[width=0.8\linewidth]{myfile.eps}
```

or

```
\usepackage[pdftex]{graphicx} ...
\includegraphics[width=0.8\linewidth]{myfile.pdf}
```

for .pdf graphics. See section 4.4 in the graphics bundle documentation (`http://www.ctan.org/tex-archive/macros/latex/required/graphics/grfguide.ps`)

A number of width problems arise when LaTeX cannot properly hyphenate a line. Please give LaTeX hyphenation hints using the `\-` command.

Broader Impact Statement

In this optional section, TMLR encourages authors to discuss possible repercussions of their work, notably any potential negative impact that a user of this research should be aware of. Authors should consult the TMLR Ethics Guidelines available on the TMLR website for guidance on how to approach this subject.

Author Contributions

If you'd like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors. Only add this information once your submission is accepted and deanonymized.

Acknowledgments

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper. Only add this information once your submission is accepted and deanonymized.

References

Amir Ali Ahmadi and Anirudha Majumdar. Some applications of polynomial optimization in operations research and real-time decision making. *Optimization Letters*, 10:709–729, 2016.

Brandon Amos, Lei Xu, and J. Zico Kolter. Input convex neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 146–155. PMLR, 06–11 Aug 2017.

Kyri Baker. Learning warm-start points for ac optimal power flow. In *2019 IEEE 29th International Workshop on Machine Learning for Signal Processing (MLSP)*, pp. 1–6. IEEE, 2019.

Stefan Bamberger, Reinhard Heckel, and Felix Krahmer. Approximating positive homogeneous functions with scale invariant neural networks. *Journal of Approximation Theory*, pp. 106177, 2025.

Dimitris Bertsimas and Bartolomeo Stellato. Online mixed-integer optimization in milliseconds. *INFORMS Journal on Computing*, 34(4):2229–2248, 2022.

Gaspard Beugnot, Julien Mairal, and Alessandro Rudi. Gloptinets: Scalable non-convex optimization with certificates, 2023. URL `https://arxiv.org/abs/2306.14932`.

Subhonmesh Bose, Dennice F Gayme, K Mani Chandy, and Steven H Low. Quadratically constrained quadratic programs on acyclic graphs with application to power flow. *IEEE Transactions on Control of Network Systems*, 2(3):278–287, 2015.

Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.

Waqquas A Bukhsh, Andreas Grothey, Ken IM McKinnon, and Paul A Trodden. Local solutions of the optimal power flow problem. *IEEE Transactions on Power Systems*, 28(4):4780–4788, 2013.

Wojciech M Czarnecki, Simon Osindero, Max Jaderberg, Grzegorz Swirszcz, and Razvan Pascanu. Sobolev training for neural networks. *Advances in neural information processing systems*, 30, 2017.

Evrim Dalkiran and Laleh Ghalami. On linear programming relaxations for solving polynomial programming problems. *Computers & Operations Research*, 99:67–77, 2018.

Yann N Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. *Advances in neural information processing systems*, 27, 2014.

Deepjyoti Deka and Sidhant Misra. Learning for dc-opf: Classifying active sets using neural nets. In *2019 IEEE Milan PowerTech*, pp. 1–6. IEEE, 2019.

Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2024. URL `https://www.gurobi.com`.

Bozhen Jiang, Qin Wang, Shengyu Wu, Yidi Wang, and Gang Lu. Advancements and future directions in the application of machine learning to ac optimal power flow: a critical review. *Energies*, 17(6):1381, 2024.

Cédric Josz and Didier Henrion. Strong duality in lasserre's hierarchy for polynomial optimization. *Optimization Letters*, 10(1):3–10, 2016.

Abhishek Kumar, Guohua Wu, Mostafa Z Ali, Rammohan Mallipeddi, Ponnuthurai Nagaratnam Suganthan, and Swagatam Das. A test-suite of non-convex constrained optimization problems from the real-world and some baseline results. *Swarm and Evolutionary Computation*, 56:100693, 2020.

Jean B Lasserre. Global optimization with polynomials and the problem of moments. *SIAM Journal on optimization*, 11(3):796–817, 2001.

Jean Bernard Lasserre. *An introduction to polynomial and semi-algebraic optimization*, volume 52. Cambridge University Press, 2015.

Javad Lavaei and Steven H Low. Zero duality gap in optimal power flow problem. *IEEE Transactions on Power Systems*, 1(27):92–107, 2012.

Alex Lemon, Anthony Man-Cho So, Yinyu Ye, et al. Low-rank semidefinite programming: Theory and applications. *Foundations and Trends® in Optimization*, 2(1-2):1–156, 2016.

Ramtin Madani, Morteza Ashraphijuo, and Javad Lavaei. Sdp solver of optimal power flow users manual, 2014.

Daniel K Molzahn, Cédric Josz, and Ian A Hiskens. Moment relaxations of optimal power flow problems: beyond the convex hull. In *2016 IEEE Global Conference on Signal and Information Processing (GlobalSIP)*, pp. 856–860. IEEE, 2016.

Katta G Murty and Santosh N Kabadi. Some np-complete problems in quadratic and nonlinear programming. Technical report, 1985.

Xiang Pan. Deepopf: deep neural networks for optimal power flow. In *Proceedings of the 8th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, pp. 250–251, 2021.

Xiang Pan, Minghua Chen, Tianyu Zhao, and Steven H Low. Deepopf: A feasibility-optimized deep neural network approach for ac optimal power flow problems. *IEEE Systems Journal*, 17(1):673–683, 2022.

Pablo A Parrilo. Semidefinite programming relaxations for semialgebraic problems. *Mathematical programming*, 96:293–320, 2003.

Power Systems Test Case Archive. Ieee Power Systems Test Case Archive. `https://labs.ece.uw.edu/pstca/`, 1993. University of Washington, Department of Electrical Engineering.

Mihai Putinar. Positive polynomials on compact semi-algebraic sets. *Indiana University Mathematics Journal*, 42(3):969–984, 1993.

Andrew Rosemberg, Mathieu Tanneau, Bruno Fanzeres, Joaquim Garcia, and Pascal Van Hentenryck. Learning optimal power flow value functions with input-convex neural networks. *Electric power systems research*, 235:110643, 2024.

Neev Samuel, Tzvi Diskin, and Ami Wiesel. Deep mimo detection. In *2017 IEEE 18th International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*, pp. 1–5. IEEE, 2017.

Hanif D Sherali and Warren P Adams. A hierarchy of relaxations between the continuous and convex hull representations for zero-one programming problems. *SIAM Journal on Discrete Mathematics*, 3(3):411–430, 1990.

Hanif D Sherali and Warren P Adams. *A reformulation-linearization technique for solving discrete and continuous nonconvex problems*, volume 31. Springer Science & Business Media, 2013.

Manish K Singh, Vassilis Kekatos, and Georgios B Giannakis. Learning to solve the ac-opf using sensitivity-informed deep neural networks. *IEEE Transactions on Power Systems*, 37(4):2833–2846, 2021.

Somayeh Sojoudi and Javad Lavaei. Exactness of semidefinite relaxations for nonlinear optimization problems with underlying graph structure. *SIAM Journal on Optimization*, 24(4):1746–1778, 2014.

Bo Sun, Jerry Huang, Nicolas Christianson, Mohammad Hajiesmaili, Adam Wierman, and Raouf Boutaba. Online algorithms with uncertainty-quantified predictions. *arXiv preprint arXiv:2310.11558*, 2023.

Pascal Van Hentenryck. Machine learning for optimal power flows. *Tutorials in Operations Research: Emerging Optimization Methods and Modeling Techniques with Applications*, pp. 62–82, 2021.

Ling Zhang and Baosen Zhang. Learning to solve the ac optimal power flow via a lagrangian approach. In *2022 North American Power Symposium (NAPS)*, pp. 1–6. IEEE, 2022.

Ling Zhang, Yize Chen, and Baosen Zhang. A convex neural network solver for dcopf with generalization guarantees. *IEEE Transactions on Control of Network Systems*, 9(2):719–730, 2021.

Fengyu Zhou and Steven H Low. Conditions for exact convex relaxation and no spurious local optima. *IEEE Transactions on Control of Network Systems*, 9(3):1468–1480, 2021.

A Additional Background

A.1 Machine Learning-Based AC-OPF Solvers

Using machine learning to assist in optimization has garnered strong interest in power systems research. Notably, several works contribute machine learning-based AC-OPF solvers, motivated by the growing need to solve large-scale problem instances with time-varying load and renewables (**??**). While practical experience and some theoretical work has shown that spurious local minima may be rare in AC-OPF, it remains an interesting and potentially important challenge to solve AC-OPF efficiently using machine learning tools (**?**). There are also several well-known examples of AC-OPF problems with spurious local solutions (**?**).

Many machine learning-based solvers, including those developed by **?** and **?**, directly map load and renewables to optimal AC-OPF solutions. **?** use a Lagrangian duality-based approach to learn solution mappings in a more favorable optimization landscape. **?** train input-convex neural networks (ICNNs) to learn the value function of parametric DC-OPF, and exploit linear programming duality to recover optimal solutions from the network predictions. **?** train neural networks to classify active constraint sets of DC-OPF problems, from which optimal solutions may be efficiently computed. **?** uses machine learning-predicted AC-OPF solutions as warm starts for local search solvers. **?** train ICNNs to learn the value function of AC-OPF, though they did not use convexification techniques to certify the optimality of AC-OPF solutions.

Though many of these approaches achieve good performance, they still lack optimality guarantees (and even feasibility guarantees in some cases). Solutions obtained from machine learning-based solvers must often be projected into the feasible set in a final step. A key challenge is that solution mappings are high dimensional and discontinuous functions that are difficult to approximate (**?**). Certifying optimality of solutions obtained from machine learning-based solvers therefore remains an important challenge.

A.2 Special Neural Network Architectures

For certain non-convex problems, the value function is convex and/or positive homogeneous. This is not always the case, even for non-convex problems which admit exact convex relaxations. However, when this is the case, it is conceivably desirable to enforce these properties in a neural network trained to approximate the value function. Therefore we turn our attention to input-convex and scale-invariant neural networks (ICNNs and SINNs), which are special neural network architectures that respectively enforce convexity and positive homogeneity in the network outputs (**??**).

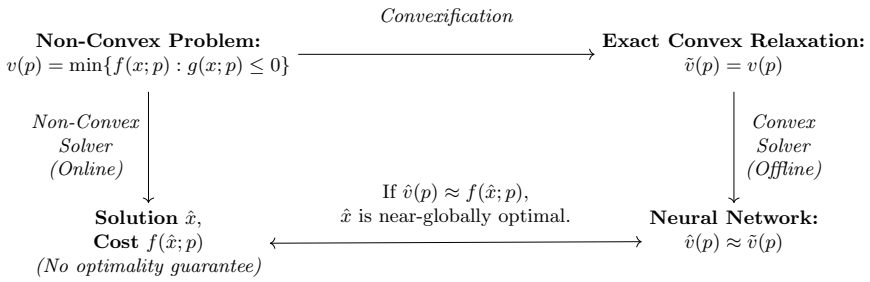

Figure 4: Neural network with skip connections.

Consider a k-layer neural network with skip connections and input $y \in \mathbb{R}^d$. For $i = 0, \ldots, k-1$ we have

$$z_{i+1} = \sigma_i \left(W_i^{(z)} z_i + W_i^{(y)} y + b_i \right), \quad h(y;\theta) = z_k$$

where z_i denotes the activation or output of layer i, $\theta = \{W_{0:k-1}^{(y)}, W_{1:k-1}^{(z)}, b_{0:k-1}\}$ are the neural network weights and biases, and σ_i are nonlinear activation functions (we define $z_0 = 0$ and $W_0^{(z)} = 0$ in the input layer). Here, $W_i^{(z)}$ are feedforward connection weights while $W_i^{(y)}$ are skip connection weights and b_i are layer biases.

By imposing certain conditions on the activation functions and weights and biases, the function $h(y;\theta)$ can be made to be convex or scale invariant in y. In particular:

1. **ICNN**: The function $h(y;\theta)$ is convex in y if all feedforward connection weights $W_{1:k-1}^{(z)}$ are non-negative and all activation functions σ_i are convex and non-decreasing (**?**). That is, for all $y_1, y_2 \in \mathbb{R}^d$, $\lambda \in [0,1]$, we have
$$h(\lambda y_1 + (1-\lambda) y_2; \theta) \leq \lambda h(y_1;\theta) + (1-\lambda) h(y_2;\theta)$$
2. **SINN**: The function $h(y;\theta)$ is scale invariant (positively homogeneous) if all biases b_i are zero and all activation functions σ_i are scale invariant (**?**). That is, for all $y \in \mathbb{R}^d$, $\lambda \geq 0$, we have
$$h(\lambda y;\theta) = \lambda h(y;\theta)$$

By enforcing these properties in the neural network architecture, ICNNs and SINNs provide strong inductive bias for learning to approximate convex or positive homogeneous functions. They may achieve lower generalization error and their outputs may be easier to interpret and integrate into optimization frameworks.

The ReLU activation function ($\mathrm{ReLU}(x) = \max\{0, x\}$) is convex, non-decreasing, and scale invariant, and therefore can be used in both ICNNs and SINNs.

B Experiment Details

B.1 Training Neural Networks

Throughout our experiments, we trained deep neural networks to approximate the value functions of non-convex problems. We used a 6-layer architecture whose input dimension d varied by experiment. The five hidden layers had progressively fewer neurons: 512, 512, 256, 64, and 16 neurons. The output layer had a single neuron. For ICNN architectures, skip connections were included between the input layer and hidden layers, while DNNs and SINNs did not have skip connections. However, the number of layers and neurons in each network was the same regardless of architecture.

To train the neural networks we used a two-stage approach: first, we trained the DNN with stochastic gradient descent (SGD) for 1000-5000 epochs, followed by 1000-5000 epochs of training with Adam. We adopted this two-stage approach because it seemed to be more stable than using either SGD or Adam by itself. The batch size during training ranged from 30-200 samples, and the total size of the data varied between 5000 and 100000 samples in our experiments. In all experiments, the data were split into training (70%) and test (30%) data sets. Our goal was to achieve zero training loss.

When training ICNNs and SINNs, the training process was modified. For SINNs, we clamped all biases b_i to zero. For ICNNs, the feedforward connection weights $W_i^{(z)}$ were clipped to be nonnegative after every training iteration. Implementing the clipping step required careful tuning of learning rates to avoid slow convergence (generally, larger learning rates were used to train ICNNs).

B.2 Power Systems Optimization

We trained neural networks to approximate the value function of AC-OPF parameterized by real and reactive load for several IEEE test power systems (see Table **??**). Our experimental procedure, which we repeated for each test system, was as follows.

For a given IEEE test power system with n buses, let $P_D^{MAX}, Q_D^{MAX} \in \mathbb{R}^n$ be vectors of the nominal real and reactive load at each bus. We randomly sampled real and reactive load vectors P_D, Q_D from $\Phi \subseteq [0, P_D^{MAX}] \times [0, Q_D^{MAX}] \subset \mathbb{R}^{2n}$ and solved the corresponding SDP relaxation of AC-OPF for each instance to obtain the relaxed optimal cost $\tilde{v}(P_D, Q_D)$. In particular, we solved the SDP relaxation in **?** using the solver in **?**. Exactness of each SDP relaxation was ascertained *a posteriori* from the rank of the obtained optimal solution. Any problem instance for which a rank-1 solution was not obtained was discarded from the data set. (These instances made up fewer than 5% of samples in all of our experiments.) The valid samples were split into training (70%) and test (30%) data sets. A deep neural network (DNN) and input-convex neural network (ICNN) were separately trained to approximate the value function $v(P_D, Q_D)$. We measured the average and maximum percentage error of the DNN and ICNN predictions over the test data set. Experiment configurations and results are reported in Table **??**.

An important detail of our experiments is that real and reactive load were not sampled from the full-dimensional parameter space. In real power systems, electricity demand at different buses is correlated, and the load data may lie in or near a low-dimensional subspace of the ambient space. For instance, in a system with n load buses, the ambient space has $2n$ dimensions, but the data may lie in an m-dimensional subspace of $\mathbb{R}^{2n}$ with $m \ll 2n$. We therefore chose to vary the intrinsic dimension of the load data in our experiments (the intrinsic data dimension in each experiment is given in the "data dim." column in Table **??**). We hypothesized that neural networks would achieve better performance on data with lower intrinsic dimension . For each test system, we tested multiple configurations of the intrinsic data dimension and number of samples. In each experiment, real and reactive load data was uniformly sampled from a randomly chosen m-dimensional subspace $\Phi \subseteq [0, P_D^{MAX}] \times [0, Q_D^{MAX}] \subset \mathbb{R}^{2n}$.

Let us now write the AC-OPF formulation. Consider a power network with a set of buses $\mathcal{N} = \{1, 2, ..., n\}$, set of generator buses $\mathcal{G} \subseteq \mathcal{N}$, and set of lines $\mathcal{L} \subseteq \mathcal{N} \times \mathcal{N}$. We define the variables, parameters, and expressions associated with bus $k \in \mathcal{N}$ as follows:

- $V_k = V_{dk} + \mathbf{j}V_{qk}$: Apparent (real and reactive) voltage.

Table 4: IEEE test systems used in our AC-OPF experiments (**?**).

IEEE Case	# Buses	# Lines	# Generators
`ieee9`	9	9	3
`ieee14`	14	20	5
`ieee30`	30	41	6
`ieee118`	118	186	54
`ieee300`	300	411	69

- $P_{Dk} + \mathbf{j}Q_{Dk}$: Real and reactive power demand or load.
- $P_{Gk} + \mathbf{j}Q_{Gk}$: Real and reactive power generation.
- $f_{Ck}(P_{Gk}) = c_{k2}P_{Gk}^2 + c_{k1}P_{Gk} + c_{k0}$: Convex quadratic cost function of real power generation.
- $f_{Vk}(V_d, V_q) = V_{dk}^2 + V_{qk}^2$: Squared voltage magnitude.

Let $\mathbf{Y} = \mathbf{G} + \mathbf{j}\mathbf{B}$ be the bus admittance matrix of an equivalent Π circuit model of the network. The real and reactive power generation at each bus are related to the bus voltages through the power flow equations $P_{Gk} := f_{Pk}(V_d, V_q)$ and $Q_{Gk} := f_{Qk}(V_d, V_q)$, where

$$f_{Pk} := P_{Dk} + V_{dk}\sum_{i=1}^{n}(\mathbf{G}_{ki}V_{di} - \mathbf{B}_{ki}V_{qi}) + V_{qk}\sum_{i=1}^{n}(\mathbf{B}_{ki}V_{di} + \mathbf{G}_{ki}V_{qi}) \tag{16}$$

$$f_{Qk} := Q_{Dk} + V_{dk}\sum_{i=1}^{n}(-\mathbf{B}_{ki}V_{di} - \mathbf{G}_{ki}V_{qi}) + V_{qk}\sum_{i=1}^{n}(\mathbf{G}_{ki}V_{di} - \mathbf{B}_{ki}V_{qi}) \tag{17}$$

We may write AC-OPF parameterized by real and reactive power demand P_D and Q_D as follows:

$$\begin{aligned} v(P_D, Q_D) := \ & \min \ \sum_{k\in\mathcal{G}} f_{Ck}(f_{Pk}(V_d, V_q)) && (18)\\ & \text{s.t. } P_{Gk}^{min} \le f_{Pk}(V_d, V_q) \le P_{Gk}^{max} && (19)\\ & \quad Q_{Gk}^{min} \le f_{Qk}(V_d, V_q) \le Q_{Gk}^{max} && (20)\\ & \quad (V_k^{min})^2 \le f_{Vk}(V_d, V_q) \le (V_k^{max})^2 && (21)\\ & \quad V_{q0} = 0 && (22) \end{aligned}$$

where constraints are defined for all buses $k \in \mathcal{N}$. Constraints equation **??** and equation **??** bound the real and reactive power output at each bus; constraint equation **??** bounds the voltage magnitude at each bus; and constraint equation **??** fixes the voltage angle of the slack bus to 0. In the full AC-OPF formulation used in some of our experiments, the magnitude of apparent power flow on lines may also be constrained. We omit these constraints here for brevity.

B.3 Digital Signal Processing

In this experiment, we consider the parametric multi-input multi-output (MIMO) detection problem:

$$\begin{aligned} v(\bar{y}) := \ & \min \ \|\bar{y} - \bar{H}x\|_2^2 && (23)\\ & \text{s.t. } x_i^2 = 1, \quad i = 1, ..., n \end{aligned}$$

Here $x \in \{-1, 1\}^n$ is the discrete signal to be detected, $\bar{H} \in \mathbb{C}^{m\times n}$ is a complex-valued channel matrix, and $\bar{y} \in \mathbb{C}^m$ is a complex-valued noisy signal of the form

$$\bar{y} = \bar{H}x + \epsilon \tag{24}$$

where ϵ is random Gaussian noise. The problem is parameterized by the received signal $\bar{y}$. Observe that any instance of the MIMO detection problem equation **??** has 2^n feasible points, each of which is a local minimum.

In our experiment, we randomly generated a fixed channel matrix $\bar{H} \in \mathbb{C}^{m \times n}$ with $m = 4$ and $n = 2$:

$$\bar{H} = \begin{bmatrix} 0.70 + \mathbf{j}0.41 & 0.40 + \mathbf{j}0.11 \\ -0.41 + \mathbf{j}0.62 & -0.80 + \mathbf{j}0.12 \\ 0.67 + \mathbf{j}0.01 & -0.02 + \mathbf{j}0.52 \\ -0.42 + \mathbf{j}0.28 & -0.28 + \mathbf{j}0.76 \end{bmatrix}$$

We generated 5000 uniform random instances of the discrete signal $x \in \{-1, 1\}^n$ and noise $\epsilon \sim \mathcal{N}(0, \sigma^2 I_m)$ with $\sigma^2 = 0.1$, from which the received signal was constructed according to equation **??**. We reformulated equation **??** as a QCQP over real variables using the transformation

$$H = \begin{bmatrix} \Re(\bar{H}) \\ \Im(\bar{H}) \end{bmatrix} \in \mathbb{R}^{2m \times n}, \quad y = \begin{bmatrix} \Re(\bar{y}) \\ \Im(\bar{y}) \end{bmatrix} \in \mathbb{R}^{2m}.$$

The transformed problem may be written as

$$\begin{aligned} v(y) = \min\ & x^T Q x + 2c^T x + r \\ \text{s.t. } & x_i^2 = 1, \quad i = 1, 2, ..., n \end{aligned} \tag{25}$$

with $Q = H^T H$, $c = -H^T y$ and $r = y^T y$. The SDP relaxation of equation **??** is

$$\begin{aligned} \tilde{v}(y) := \min\ & \langle M, X \rangle \\ \text{s.t. } & X_{ii} = 1, \quad i = 1, 2, ..., n+1 \\ & X \succeq 0 \end{aligned} \tag{26}$$

where we have implicitly lifted x to $[x^T \quad 1]^T$ and where

$$M = \begin{bmatrix} Q & c \\ c^T & r \end{bmatrix}$$

We solved the SDP relaxation equation **??** in our experiments to evaluate $\tilde{v}(y)$.

B.4 Integer Optimization

In this experiment, we consider the parametric 0-1 knapsack problem with n items:

$$\begin{aligned} v(c) := \ \max\ & c^T x \\ \text{s.t. } & w^T x \leq b \\ & x \in \{0, 1\}^n \end{aligned} \tag{27}$$

Here $c \in \mathbb{R}^n_+$ is the cost vector, $w \in \mathbb{R}^n_+$ is the weight vector, and $b \in \mathbb{R}_+$ is the knapsack capacity or budget. The problem is parameterized by the cost vector c.

We solved tens of thousands of instances of problem equation **??** to develop training data sets for our experiments. We considered problems with $n = 10$ and $n = 30$ items. Cost vectors were sampled from a subspace of the ambient parameter space $\Phi = [0, 10]^n \subset \mathbb{R}^n_+$. We followed a training procedure similar to our other experiments: data were split into training (70%) and test (30%) data sets, and we used both SGD and Adam to train the neural networks for 6000 epochs. We trained a DNN, ICNN, as well as SINN and CSINN (scale-invariant and convex scale-invariant neural network) in each experiment. We measured the average and maximum test set error of each neural network. For solving instances of the 0-1 knapsack problem, we did not explicitly use convexification techniques. However, many state-of-the-art MILP solvers do in fact benefit from convexification "under the hood" – for instance, by using lift-and-project methods to speed branch-and-bound.

Note that the value function $v(c)$ in equation **??** is convex in c, as it is a maximum over a finite number of linear functions in $\mathbb{R}^n$. It is also scale invariant, as scaling the cost vector by a positive constant does not change the optimal solution and only proportionally scales the optimal objective function value. We therefore explored training DNNs, ICNNs, SINNs, and CSINNs to learn the value function $v(c)$ in this experiment.

In this experiment, we also explored the potential to obtain optimal solution mappings from the trained neural networks. Since $v(c)$ is scale-invariant, it satisfies the equation $v(\lambda c) = \lambda v(c)$ for all $\lambda \geq 0$. Therefore,

$$v(c) = \nabla v(c)^T c$$

for all $c \in \mathbb{R}^n_+$ where $\nabla v(c)$ is well-defined (i.e., when equation **??** has a unique solution $x^*(c) \in \{0,1\}^n$). Therefore, for such a $c \in \mathbb{R}^n_+$, it can be seen that $\nabla v(c) = x^*(c)$. Therefore, if the gradient of a trained neural network $\hat{v}(c)$ approximates the gradient of $v(c)$ well, then $\nabla \hat{v}(c)$ may approximate an optimal solution mapping for equation **??**. We conducted experiments in which we used the gradient of the trained CSINN $\hat{v}(c)$ to obtain approximate 0-1 knapsack solutions by rounding / projecting $\nabla \hat{v}(c)$ to the feasible set of equation **??**. We were able to recover optimal solutions in over 70% of test problem instances for the 10-item knapsack problem. This method could likely be further improved via Sobolev training of the CSINN, which is a method of controlling gradient approximation error during training (**?**).

C Approximation Error Bounds and ICNN Performance Guarantees

There are two ways in which we may bound the approximation error of $\hat{v}(p)$: one-sided and two-sided error bounds.

Definition C.1. A ***one-sided error bound*** only bounds the error in one direction. It is a bound of the form

$$\hat{v}(p) - v(p) \leq B \quad \forall\, p \in \Phi.$$

Given this one-sided error bound, one may always conclude that if $\hat{x}$ is feasible for equation **??** then

$$f(\hat{x}; p) - v(p) \leq f(\hat{x}; p) - \hat{v}(p) + B.$$

Definition C.2. A ***two-sided error bound*** bounds the error in both directions, or the absolute error. It is a bound of the form

$$|\hat{v}(p) - v(p)| \leq B \quad \forall\, p \in \Phi.$$

Given this two-sided error bound, one may always conclude that if $\hat{x}$ is feasible for equation **??** then

$$f(\hat{x}; p) - \hat{v}(p) - B \leq f(\hat{x}; p) - v(p) \leq f(\hat{x}; p) - \hat{v}(p) + B.$$

In the context of optimality certification, a one-sided error bound allows one to bound the optimality gap from *above*, while a two-sided error bound allows one to bound the optimality gap from *above* and *below*. (The optimality gap is always bounded below by zero as well. However, it is possible to provide a tighter lower bound with a two-sided error bound.)

As we have stated, the ability of a neural network to certify optimality depends on its ability to approximate the value function well. We introduced the notions of one-sided and two-sided error bounds, which allow one to bound the optimality gap of feasible solutions from one or both directions using the neural network's predictions.

Unfortunately, the gap between theory and observation remains quite large when it comes to bounding the error of generic deep neural networks (DNNs). By comparison, input-convex neural networks (ICNNs) (**?**) can have much stronger theoretical performance guarantees for learning to approximate convex functions. We therefore turn our attention to approximating *convex* value functions with ICNNs. We note that the value functions of non-convex problems can be (provably) convex. For instance, if a QCQP parameterized by right-hand side uncertainty always admits an exact SDP relaxation, then its value function is convex (see Appendix **??** for more details).

? studied the approximation abilities of ICNNs in the context of learning to approximate the value function of parametric DC-OPF. Furthermore, **?** developed a two-sided error bound for ICNNs trained to approximate the value function of a parametric linear program (LP). They show that the the error is bounded over the convex hull of the training data if the ICNN achieves zero value error *and* zero gradient error during training. We derive a similar two-sided error bound but under the weaker assumption of *bounded* gradient error (Assumption **??** below). We also derive a one-sided error bound when no assumption about the gradient

error is made. This is an important generalization, as zero gradient error over the training data is usually not achieved even when zero value error is achieved.

Notation, Assumptions, and Error Bounds

Let $v(p)$ denote a convex value function, and $\hat{v}(p)$ denote an ICNN trained to approximate the value function. Note that problem equation **??** need not be convex for it to have a convex value function. Let $\mathcal{P} = \{p_1, ..., p_N\} \subset \Phi$ denote the training data, with corresponding labels $\{v(p_1), ..., v(p_N)\} \subset \mathbb{R}$. Let $\text{conv}(\cdot)$ denote the convex hull of a set, and let $\text{diam}(\cdot)$ denote the diameter of a set. All norms $\|\cdot\|$ denote the Euclidean norm.

Furthermore, we define the "training data simplex" of $p \in \text{conv}(\mathcal{P})$ as follows. The training data simplex $\mathcal{S}(p) \subset \mathcal{P}$ is the minimum diameter subset of the training data that contains p in its convex hull. That is,

$$\mathcal{S}(p) = \arg\min_{\mathcal{S} \subseteq \mathcal{P}} \{\text{diam}(\mathcal{S}) : \text{p} \in \text{conv}(\mathcal{S}\}.$$

We assume that $\mathcal{S}(p)$ is well defined for all $p \in \text{conv}(\mathcal{P})$ (i.e., there are never ties between subsets of the data). As a corollary of Caratheodory's theorem, $\mathcal{S}(p)$ contains at most $d+1$ training data points.

We now state our main error bound results. Note that these results hold only for points $p \in \text{conv}(\mathcal{P})$, and not arbitrary $p \in \Phi$. This is a common limitation of theoretical performance guarantees for ICNNs, and is difficult to avoid without making stronger assumptions.

Our first error bound uses the following assumption of zero approximation error over the training data.

Assumption C.3 (*Zero Value Error*)**.** The trained neural network approximates the value function with zero error over the training data set. That is,

$$\hat{v}(p_i) = v(p_i), \quad i = 1, 2, ..., N.$$

Under this assumption, we obtain the following one-sided error bound for $\hat{v}(p)$.

Theorem C.4 (*One-Sided Error Bound*)**.** *Let $v(p)$ be a convex value function and $\hat{v}(p)$ be an ICNN trained to approximate $v(p)$. If Assumption **??** holds, then for any $p \in \text{conv}(\mathcal{P})$ we have*

$$\hat{v}(p) - v(p) \leq \max_{p_i \in \mathcal{S}(p)} \|\nabla v(p_i)\| \, \text{Diam}(\mathcal{S}(\text{p}))$$

where $\mathcal{S}(p)$ is the training data simplex of p.

We strengthen this result by assuming bounded error of the gradient of $\hat{v}(p)$ over the training data.

Assumption C.5 (*Bounded Gradient Error*)**.** The gradient of the trained neural network approximates the gradient of the value function over the training data set with error ϵ_i such that

$$\nabla \hat{v}(p_i) = \nabla v(p_i) + \epsilon_i, \quad i = 1, 2, ..., N,$$

where

$$\|\epsilon_i\| \leq r \|\nabla v(p_i)\|, \quad i = 1, 2, ..., N,$$

for some scalar $r \geq 0$.

Under this additional assumption, we obtain the following two-sided error bound for $\hat{v}(p)$.

Theorem C.6 (*Two-Sided Error Bound*)**.** *Let $v(p)$ be a convex value function and $\hat{v}(p)$ be an ICNN trained to approximate $v(p)$. If Assumptions **??** and **??** hold, then for any $p \in \text{conv}(\mathcal{P})$ we have*

$$|\hat{v}(p) - v(p)| \leq (1 + r) \max_{p_i \in \mathcal{S}(p)} \|\nabla v(p_i)\| \, \text{Diam}(\mathcal{S}(\text{p}))$$

where $\mathcal{S}(p)$ is the training data simplex of p.

Theorems **??** and **??** are illuminating, as they suggest that one should draw more training samples (and reduce the simplex diameter) in regions where the gradient is large to improve ICNN performance.

C.1 Proof of Theorem ??

Suppose $p \in \text{conv}(\mathcal{P})$. Then $\mathcal{S}(p) = \{p_1, ..., p_k\}$ is well-defined and contains $k \leq d+1$ training data points. Since $p \in \text{conv}(\mathcal{S}(p))$ by definition, there exist $\lambda_1, ..., \lambda_k$ such that

$$\sum_{i=1}^{k} \lambda_i p_i = p, \tag{28}$$

$$\sum_{i=1}^{k} \lambda_i = 1, \text{ and} \tag{29}$$

$$\lambda_1, ..., \lambda_k \geq 0. \tag{30}$$

From convexity of $\hat{v}(p)$ and Assumption **??**, we obtain the upper bound

$$\hat{v}(p) \leq \sum_{i=1}^{k} \lambda_i v(p_i)$$

from which we obtain the trivial one-sided error bound:

$$\begin{aligned} \hat{v}(p) - v(p) &\leq \sum_{i=1}^{k} \lambda_i v(p_i) - v(p) \\ &= \sum_{i=1}^{k} \lambda_i (v(p_i) - v(p)). \end{aligned}$$

From convexity of $v(p)$ we also have that

$$\begin{aligned} v(p_i) - v(p) &\leq \nabla v(p_i)^T (p_i - p) \\ &\leq \|\nabla v(p_i)\| \|p_i - p\|. \end{aligned}$$

for all $p_i \in \mathcal{S}(p)$. Substituting back into the previous inequality and maximizing over $p_i \in \mathcal{S}(p)$, we obtain the desired result:

$$\begin{aligned} \hat{v}(p) - v(p) \leq & \sum_{i=1}^{k} \lambda_i \|\nabla v(p_i)\| \|p_i - p\| \\ &\leq \max_{p_i \in \mathcal{S}(p)} \|\nabla v(p_i)\| \|p_i - p\| \\ &\leq \max_{p_i \in \mathcal{S}(p)} \|\nabla v(p_i)\| \operatorname{diam}(\mathcal{S}(p)). \quad \square \end{aligned}$$

C.2 Proof of Theorem ??

The proof closely follows the proof of theorem **??**, but we make use of the subgradient inequality for convex functions to obtain a two-sided error bound.

Suppose $p \in \text{conv}(\mathcal{P})$. Then $\mathcal{S}(p) = \{p_1, ..., p_k\}$ is well-defined and contains $k \leq d+1$ training data points. Since $p \in \text{conv}(\mathcal{S}(p))$ by definition, there exist $\lambda_1, ..., \lambda_k$ such that

$$\sum_{i=1}^{k} \lambda_i p_i = p, \tag{31}$$

$$\sum_{i=1}^{k} \lambda_i = 1, \text{ and} \tag{32}$$

$$\lambda_1, ..., \lambda_k \geq 0. \tag{33}$$

From convexity of $v(p)$ and $\hat{v}(p)$ and Assumption **??** we obtain the upper bound

$$v(p), \hat{v}(p) \leq \sum_{i=1}^{k} \lambda_i v(p_i).$$

Furthermore, for any $p_i \in \mathcal{S}(p)$ we also have by convexity that

$$v(p) \geq v(p_i) + \nabla v(p_i)^T (p - p_i)$$

and

$$\hat{v}(p) \geq \hat{v}(p_i) + \nabla \hat{v}(p_i)^T (p - p_i).$$

Therefore,

$$v(p), \hat{v}(p) \geq \min\{v(p_i) + \nabla v(p_i)^T (p - p_i),\ \hat{v}(p_i) + \nabla \hat{v}(p_i)^T (p - p_i)\}.$$

Combining these inequalities, we obtain for each $p_i \in \mathcal{S}(p)$ the two-sided error bound

$$\begin{aligned}
|\hat{v}(p) - v(p)| &\leq \sum_{j=1}^{k} \lambda_j v(p_j) - \min\{v(p_i) + \nabla v(p_i)^T (p - p_i),\ \hat{v}(p_i) + \nabla \hat{v}(p_i)^T (p - p_i)\} \\
&= \sum_{j=1}^{k} \lambda_j v(p_j) - v(p_i) + \max\{\nabla v(p_i)^T (p_i - p),\ \nabla \hat{v}(p_i)^T (p_i - p)\}.
\end{aligned}$$

Summing these error bounds over $p_i \in \mathcal{S}(p)$ with weights given by $\lambda_1, ..., \lambda_k$, we get

$$\begin{aligned}
|\hat{v}(p) - v(p)| &\leq \sum_{j=1}^{k} \lambda_j v(p_j) - \sum_{i=1}^{k} \lambda_i v(p_i) + \sum_{i=1}^{k} \lambda_i \max\{\nabla v(p_i)^T (p_i - p),\ \nabla \hat{v}(p_i)^T (p_i - p)\} \\
&= \sum_{i=1}^{k} \lambda_i \max\{\nabla v(p_i)^T (p_i - p),\ \nabla \hat{v}(p_i)^T (p_i - p)\} \\
&\leq \max_{p_i \in \mathcal{S}(p)} \{\max\{\nabla v(p_i)^T (p_i - p),\ \nabla \hat{v}(p_i)^T (p_i - p)\}\},
\end{aligned}$$

where we have maximized over $p_i \in \mathcal{S}(p)$ in the final step. Now we use the identity

$$\max\{a, b\} = \frac{a + b + |a - b|}{2}$$

to obtain

$$\begin{aligned}
\max\{\nabla v(p_i)^T (p_i - p),\ \nabla \hat{v}(p_i)^T (p_i - p)\} &= \frac{(\nabla v(p_i) + \nabla v(p_i) + \epsilon_i)^T (p_i - p) + |\epsilon_i^T (p_i - p)|}{2} \\
&\leq \frac{2\|\nabla v(p_i)\|\|p - p_i\| + 2\|\epsilon_i\|\|p - p_i\|}{2} \\
&= (\|\nabla v(p_i)\| + \|\epsilon_i\|)\|p - p_i\| \\
&\leq (1 + r)\|\nabla v(p_i)\|\|p - p_i\|.
\end{aligned}$$

We have used Assumption **??** to decompose the gradient $\nabla \hat{v}(p_i)$ and bound norm of the error $\|\epsilon_i\|$. Substituting back into the previous inequality, we obtain the desired result:

$$|\hat{v}(p) - v(p)| \leq (1 + r) \max_{p_i \in \mathcal{S}(p)} \|\nabla v(p_i)\| \operatorname{diam}(\mathcal{S}(p)). \quad \square$$

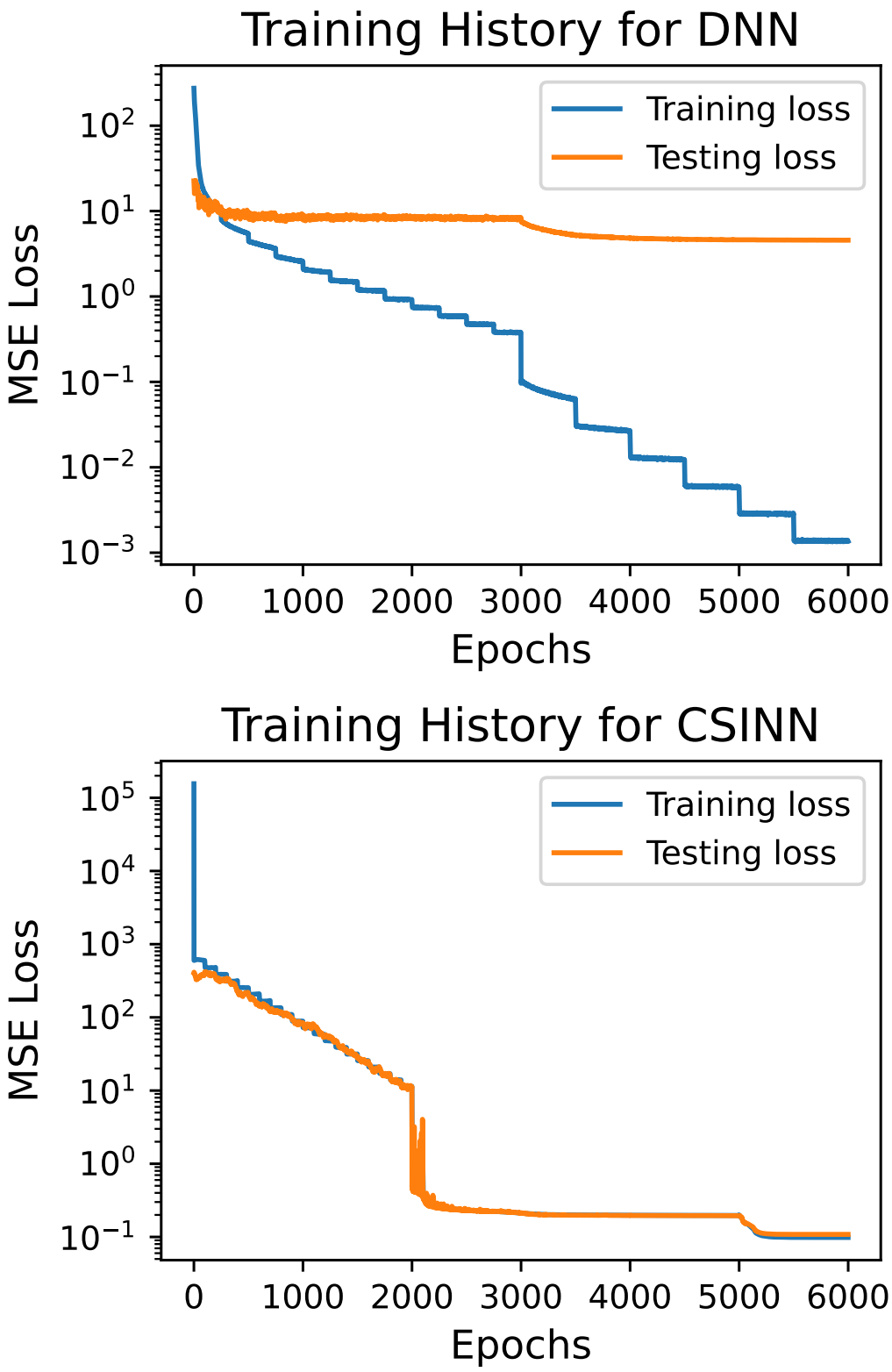

Figure 1: Our proposed approach for neural network-based optimality certification.

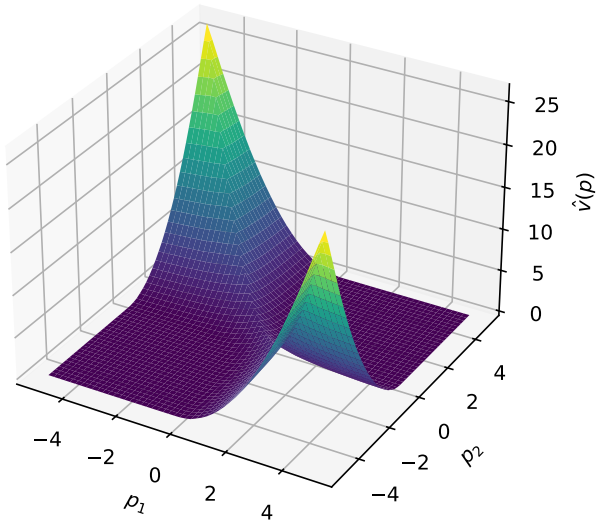

Figure 2: IPM convergence behavior for problem equation **??** for $p = (-2, 1)$. The global minimum is near $(-2.3, -0.5)$, while a spurious local minimum is near $(0.6, 1.8)$.

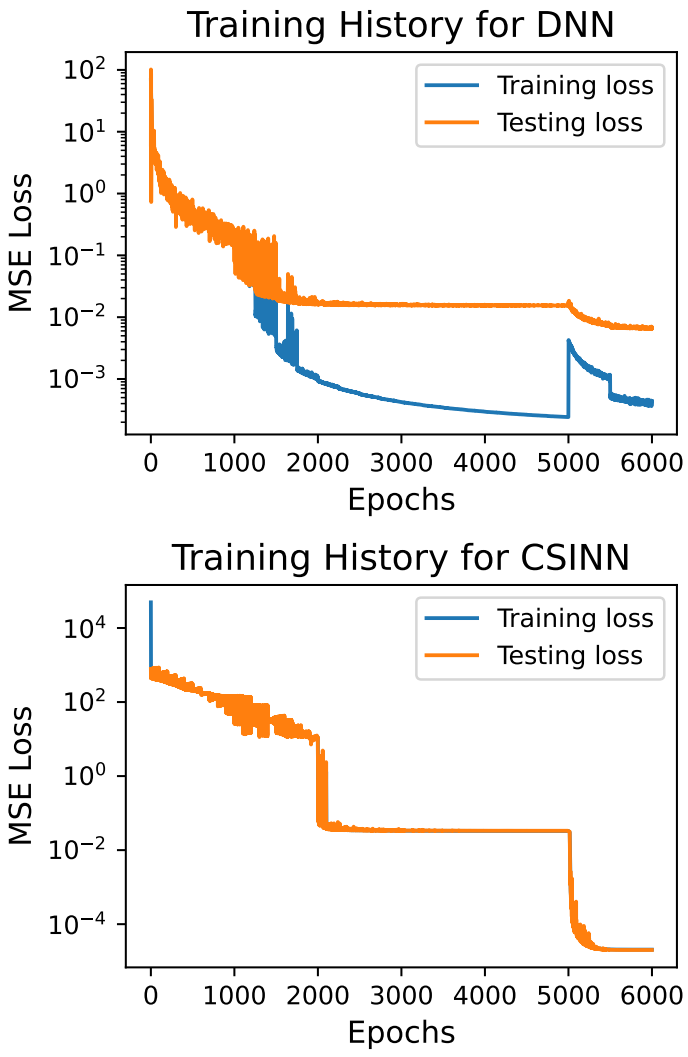

Figure 3: Neural network-based value function predictions over target parameter subspace $\Phi = [-5, 5] \times [-5, 5]$ for problem equation **??**.